**LAPORAN PRAKTIKUM**

**STRUKTUR DATA**

**QUEUE**

**Disusun Oleh:**

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

**DAPERTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**TAHUN 2026**

# KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data dengan materi Queue ini dengan baik. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum serta untuk memperdalam pemahaman mengenai konsep Queue, prinsip FIFO (First In First Out), dan penerapannya dalam pemrograman. Dalam proses penyusunan laporan ini, penulis juga memperoleh bantuan dan dukungan dari berbagai pihak, baik dosen pengampu maupun teman-teman, sehingga laporan ini dapat diselesaikan tepat waktu.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca dalam memahami materi Queue pada struktur data.

Padang, 2026

Penulis

## DAFTAR ISI

**KATA PENGANTAR** ..... **i**
**DAFTAR ISI** ..... **ii**
**BAB I PENDAHULUAN** ..... **1**
1.1 Latar Belakang ..... 1
1.2 Tujuan Praktikum ..... 1
1.3 Manfaat Praktikum ..... 1
**BAB II PEMBAHASAN** ..... **2**
2.1 Dasar Teori ..... 2
2.2 Praktikum Class “QueueArray” ..... 2
2.3 Praktikum class “QueueArrayDriver” ..... 6
2.4 Praktikum class “QueueLinkedList” ..... 9
2.5 Praktikum class “ReverseData” ..... 11
2.6 Praktikum class “IterasiQueue” ..... 14
2.7 Tugas Praktikum ..... 16
**BAB III PENUTUP** ..... **27**
3.1 Kesimpulan ..... 27
**DAFTAR PUSTAKA** ..... **28**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam dunia pemrograman yang digunakan untuk mengorganisir dan mengelola data agar lebih efisien. Salah satu jenis struktur data yang sering digunakan adalah Queue. Queue adalah struktur data yang menerapkan prinsip FIFO (First In First Out), di mana data yang pertama kali masuk akan menjadi data yang pertama kali keluar. Konsep ini sangat relevan dengan berbagai aktivitas sehari-hari, seperti antrian di loket pelayanan, kasir, maupun sistem penjadwalan pada komputer.

Pemahaman mengenai Queue sangat penting bagi mahasiswa dalam mengembangkan kemampuan logika pemrograman dan pemecahan masalah. Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Queue, cara kerjanya, serta mampu mengimplementasikannya dalam bentuk program. Dengan demikian, pengetahuan yang diperoleh tidak hanya bersifat teoritis, tetapi juga dapat diterapkan secara praktis dalam berbagai kasus di dunia nyata.

## 1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Queue.
2. Mengetahui prinsip kerja Queue (FIFO).
3. Mampu mengimplementasikan Queue dalam bahasa pemrograman.
4. Memahami operasi dasar pada Queue seperti enqueue, dequeue, dan peek.

## 1.3 Manfaat Praktikum

1. Menambah pemahaman tentang struktur data Queue.
2. Melatih kemampuan dalam mengimplementasikan konsep Queue ke dalam program.
3. Membantu memahami penerapan Queue dalam kehidupan nyata maupun sistem komputer.
4. Meningkatkan logika dan kemampuan problem solving dalam pemrograman.

# BAB II
# PEMBAHASAN

## 2.1 Dasar Teori

Queue merupakan salah satu struktur data linear yang menggunakan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Konsep Queue banyak dijumpai dalam kehidupan sehari-hari, seperti antrian di bank, rumah sakit, maupun sistem layanan lainnya. Dalam dunia komputer, Queue digunakan untuk mengatur proses secara berurutan, seperti pada antrian proses di CPU, antrian pencetakan dokumen, dan pengelolaan data dalam sistem.

Operasi dasar Queue

- *Enqueue*: Menambahkan elemen ke belakang antrian.
- *Dequeue*: Menghapus elemen dari depan antrian.
- *Peek/Front*: Melihat elemen terdepan tanpa menghapusnya.

Implementasi Queue

- Menggunakan Array.
- Menggunakan Linked List.

## 2.2 Praktikum Class "QueueArray"

```java
package pekan4_2511533014;

public class QueueArray_2511533014 {
    int front_3014, rear_3014, size_3014;
    int capacity_3014;
    int array_3014[];

    public QueueArray_2511533014(int capacity) {
        this.capacity_3014 = capacity;
        front_3014 = this.size_3014 = 0;
        rear_3014 = capacity - 1;
        array_3014 = new int [this.capacity_3014];

    }
    boolean isFull_3014 (QueueArray_2511533014 queue) {
        return (queue.size_3014 == queue.capacity_3014);
    }
    boolean isEmpty_3014 (QueueArray_2511533014 queue) {
        return (queue.size_3014 == 0);
    }
    void enqueue_3014 (int item) {
        if (isFull_3014(this))
            return;
        this.rear_3014 = (this.rear_3014 + 1) % this.capacity_3014;
        this.array_3014 [this.rear_3014] = item;
        this.size_3014 = this.size_3014 + 1;
        System.out.println(item + " enqueued to queue");
```

```java
    int dequeue_3014() {
        if (isEmpty_3014(this))
            return Integer.MIN_VALUE;
        int item = this.array_3014[this.front_3014];
        this.front_3014 = (this.front_3014 + 1) % this.capacity_3014;
        this.size_3014 = this.size_3014 - 1;
        return item;
    }

    int front_3014() {
        if (isEmpty_3014(this))
            return Integer.MIN_VALUE;
        return this.array_3014[this.front_3014];
    }

    int rear_3014() {
        if (isEmpty_3014(this))
            return Integer.MIN_VALUE;
        return this.array_3014[this.rear_3014];
    }

    // mencetak elemen antrian
    void display_3014() {
        int i;
        if (front_3014 == rear_3014) {
            System.out.println("\nAntrian Kosong\n");
            return;
        }
```

```java

        // kunjungi dari belakang dan cetak
        for (i = front_3014; i < rear_3014; i++)
            System.out.printf(" %d <-- ", array_3014[i]);
        return;

    }
    }
```

**Gambar 2.1 Kode Program Praktikum QueueArray**

Program ini merupakan implementasi struktur data *queue* (antrian) menggunakan array pada bahasa pemrograman Java. Konsep yang digunakan adalah *FIFO* (*First In First Out*), di mana elemen yang pertama masuk akan menjadi yang pertama keluar. Program ini memiliki atribut untuk menyimpan posisi depan (*front*), belakang (*rear*), ukuran antrian, kapasitas maksimum, serta array sebagai tempat penyimpanan data. Selain itu, terdapat method untuk mengecek apakah antrian penuh atau kosong, menambah data (*enqueue*), menghapus data (*dequeue*), melihat elemen terdepan dan terakhir, serta menampilkan isi antrian.

Berikut penjelasan langkah-langkah kerjanya:

1. **int front_3014, rear_3014, size_3014;**

**int capacity_3014;**

**int array_3014[];**

→ Mendeklarasikan variabel:

- front_3014: posisi depan antrian
- rear_3014: posisi belakang antrian
- size_3014: jumlah elemen saat ini
- capacity_3014: kapasitas maksimum
- array_3014[]: array penyimpan data

2. **public QueueArray_2511533014(int capacity) {**

   → Constructor untuk inisialisasi objek queue.

3. **this.capacity_3014 = capacity;**

   → Mengisi kapasitas queue sesuai input.

4. **front_3014 = this.size_3014 = 0;**

   → Mengatur posisi awal front dan size menjadi 0.

5. **rear_3014 = capacity - 1;**

   → Posisi rear di-set di akhir array (untuk circular queue).

6. **array_3014 = new int [this.capacity_3014];**

   → Membuat array dengan kapasitas tertentu.

7. **boolean isFull_3014 (QueueArray_2511533014 queue) {**

   → Method untuk mengecek apakah queue penuh.

8. **return (queue.size_3014 == queue.capacity_3014);**

   → Mengembalikan true jika size = capacity.

9. **boolean isEmpty_3014 (QueueArray_2511533014 queue) {**

   → Method untuk mengecek apakah queue kosong.

10. **return (queue.size_3014 == 0);**

    → Mengembalikan true jika tidak ada elemen.

11. **void enqueue_3014 (int item) {**

    → Method untuk menambahkan elemen ke queue.

12. **if (isFull_3014(this))**
    **return;**
    → Jika penuh, proses dihentikan.
13. **this.rear_3014 = (this.rear_3014 + 1) % this.capacity_3014;**
    → Menggeser posisi rear (circular queue).
14. **this.array_3014 [this.rear_3014] = item;**
    → Menyimpan item ke array.
15. **this.size_3014 = this.size_3014 + 1;**
    → Menambah jumlah elemen.
16. **System.out.println(item + " enqueued to queue");**
    → Menampilkan pesan bahwa data berhasil ditambahkan.
17. **int dequeue_3014() {**
    → Method untuk menghapus elemen dari queue.
18. **if (isEmpty_3014(this))**
    **return Integer.MIN_VALUE;**
    → Jika kosong, mengembalikan nilai minimum integer.
19. int item = this.array_3014[this.front_3014];
    → Mengambil elemen paling depan.
20. this.front_3014 = (this.front_3014 + 1) % this.capacity_3014;
    → Menggeser posisi front.
21. this.size_3014 = this.size_3014 - 1;
    → Mengurangi jumlah elemen.
22. return item;
    → Mengembalikan nilai yang dihapus.
23. int front_3014() {
    → Method untuk melihat elemen terdepan.
24. if (isEmpty_3014(this))
    return Integer.MIN_VALUE;
    → Jika kosong, return nilai minimum.
25. return this.array_3014[this.front_3014];
    → Mengembalikan elemen depan.
26. int rear_3014() {

→ Method untuk melihat elemen terakhir.

27. if (isEmpty_3014(this))

    return Integer.MIN_VALUE;

    → Jika kosong, return nilai minimum.

28. return this.array_3014[this.rear_3014];

    → Mengembalikan elemen belakang.

29. void display_3014() {

    → Method untuk menampilkan isi queue.

30. int i;

    → Variabel bantu untuk looping.

31. if (front_3014 == rear_3014) {

    → Mengecek apakah queue kosong (logika sederhana).

32. System.out.println("\nAntrian Kosong\n");

    return;

    → Menampilkan pesan jika kosong.

33. for (i = front_3014; i < rear_3014; i++)

    → Perulangan dari front ke rear.

34. System.out.printf(" %d <-- ", array_3014[i]);

    → Menampilkan elemen queue.

35. return;

    → Mengakhiri method.

## 2.3 Praktikum class "QueueArrayDriver"

```java
package pekan4_2511533014;

public class QueueArrayDriver_2511533014 {
        public static void main(String[] args) {
            QueueArray_2511533014 queue = new QueueArray_2511533014(1000);
            queue.enqueue_3014(10);
            queue.enqueue_3014(20);
            queue.enqueue_3014(30);
            queue.enqueue_3014(40);

            System.out.println("Item di depan " + queue.front_3014());
            System.out.println("Item paling belakang " + queue.rear_3014());
            System.out.println("Tampilan queue");
            queue.display_3014();
            System.out.println();

            System.out.println(queue.dequeue_3014() + " dihapus dari queue");
            System.out.println("item di depan: " + queue.front_3014());
            System.out.println("item dibelakang: " + queue.rear_3014());
            System.out.println("tampilan queue setelah satu data dihapus");
            queue.display_3014();
        }
    }
```

**Gambar 2.2 Kode Program praktikum QueueArrayDriver**

Program ini merupakan *driver class* (program utama) dalam □ahasa Java yang digunakan untuk menguji implementasi struktur data *queue* berbasis array. Di dalam program, dibuat sebuah objek queue dengan kapasitas tertentu, kemudian dilakukan operasi penambahan data (*enqueue*), melihat elemen depan dan belakang, menampilkan isi queue, serta menghapus elemen (*dequeue*). Program ini menunjukkan bagaimana konsep *FIFO (First In First Out)* bekerja, yaitu data yang pertama masuk akan menjadi yang pertama keluar.

Berikut penjelasan langkah-langkah kerjanya:

1. public static void main(String[] args) {
   → Method utama (*main method*) sebagai titik awal eksekusi program.
2. QueueArray_2511533014 queue = new QueueArray_2511533014(1000);
   → Membuat objek queue dengan kapasitas 1000 elemen.
3. queue.enqueue_3014(10);
   queue.enqueue_3014(20);
   queue.enqueue_3014(30);
   queue.enqueue_3014(40);
   → Menambahkan data ke dalam queue: 10, 20, 30, dan 40 (urutan masuk).
4. System.out.println("Item di depan " + queue.front_3014());
   → Menampilkan elemen paling depan (data pertama yang masuk).
5. System.out.println("Item paling belakang " + queue.rear_3014());
   → Menampilkan elemen paling belakang (data terakhir yang masuk).
6. System.out.println("Tampilan queue");
   → Menampilkan teks judul sebelum isi queue.
7. queue.display_3014();
   → Memanggil method untuk menampilkan seluruh isi queue.
8. System.out.println();

   → Memberi baris kosong agar output lebih rapi.

9. System.out.println(queue.dequeue_3014() + " dihapus dari queue");
   → Menghapus elemen paling depan dari queue dan menampilkannya.
10. System.out.println("item di depan: " + queue.front_3014());

→ Menampilkan elemen depan setelah penghapusan.

11. System.out.println("item dibelakang: " + queue.rear_3014());

    → Menampilkan elemen belakang setelah penghapusan.

12. System.out.println("tampilan queue setelah satu data dihapus");

    → Menampilkan teks penjelasan setelah operasi dequeue.

13. queue.display_3014();

    → Menampilkan isi queue terbaru setelah satu data dihapus.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.3 dibawah ini.

```
<terminated> QueueArrayDriver_2511533014 [Java Appli
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
Tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan: 20
item dibelakang: 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

**Gambar 2.3 Output kode program QueueArrayDriver**

### 2.3.1 Analisis

Output tersebut menunjukkan bahwa program *queue* berbasis array telah berjalan dengan benar sesuai konsep *First In First Out (FIFO)*. Pada awalnya, data 10, 20, 30, dan 40 berhasil dimasukkan ke dalam queue, yang ditandai dengan pesan "enqueued to queue", di mana 10 menjadi elemen paling depan dan 40 sebagai elemen paling belakang. Ketika ditampilkan, isi queue menunjukkan urutan sesuai data masuk yaitu 10 ← 20 ← 30. Selanjutnya, saat dilakukan operasi *dequeue*, elemen paling depan yaitu 10 dihapus dari queue. Setelah penghapusan,

elemen terdepan berubah menjadi 20 dan elemen paling belakang tetap 40. Tampilan queue terakhir menunjukkan bahwa data tersisa adalah 20 ← 30, yang menandakan bahwa proses antrian berjalan sesuai prinsip FIFO di mana data yang pertama masuk akan menjadi yang pertama keluar.

### 2.4 Praktikum class "QueueLinkedList"

```java
package pekan4_2511533014;
import java.util.LinkedList;
import java.util.Queue;
public class QueueLinkedList_2511533014 {
    public static void main (String[] args) {
        Queue<Integer> q = new LinkedList<>();
        //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i = 0; i < 6; i++)
            q.add(i);
        //Menampilkan isi antrian.
        System.out.println("Elemen Antrian " + q);
        //Untuk menghapus kepala antrian.
        int hapus = q.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q);
        //Untuk melihat antrian terdepan
        int depan = q.peek();
        System.out.println("Kepala Antrian = " + depan);

        int banyak = q.size();
        System.out.println("Size Antrian = " + banyak);


        }

}
```

**Gambar 2.4 Kode program praktikum class QueueLinkedList**

Program ini merupakan implementasi struktur data *queue* menggunakan Java dengan bantuan kelas LinkedList yang mengimplementasikan interface Queue. Program berfungsi untuk menambahkan beberapa elemen ke dalam antrian, menampilkan isi antrian, menghapus elemen terdepan, melihat elemen paling depan tanpa menghapusnya, serta mengetahui jumlah elemen dalam antrian. Semua operasi ini menunjukkan konsep *FIFO (First In First Out)*, di mana elemen yang pertama masuk akan menjadi yang pertama keluar.

Berikut penjelasan langkah-langkah kerjanya:

1. import java.util.LinkedList;
   import java.util.Queue;

→ Mengimpor library:

- LinkedList sebagai implementasi queue
- Queue sebagai interface antrian

2. public static void main (String[] args) {

   → Method utama sebagai titik awal program.

3. Queue<Integer> q = new LinkedList<>();

   → Membuat objek queue bernama q dengan tipe data Integer menggunakan LinkedList.

4. for (int i = 0; i < 6; i++)
   q.add(i);

   → Perulangan untuk menambahkan elemen 0 sampai 5 ke dalam queue.

5. System.out.println("Elemen Antrian " + q);

   → Menampilkan seluruh isi queue (akan tampil [0, 1, 2, 3, 4, 5]).

6. int hapus = q.remove();

   → Menghapus elemen paling depan dari queue (yaitu 0) dan menyimpannya ke variabel hapus.

7. System.out.println("Hapus elemen = " + hapus);

   → Menampilkan elemen yang dihapus.

8. System.out.println(q);

   → Menampilkan isi queue setelah penghapusan (menjadi [1, 2, 3, 4, 5]).

9. int depan = q.peek();

   → Mengambil elemen paling depan tanpa menghapusnya.

10. System.out.println("Kepala Antrian = " + depan);

    → Menampilkan elemen terdepan (yaitu 1).

11. int banyak = q.size();

    → Menghitung jumlah elemen dalam queue.

12. System.out.println("Size Antrian = " + banyak);

    → Menampilkan jumlah elemen queue (5).

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.5 dibawah ini.

```
<terminated> QueueLinkedList_2511533014 [Jav
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

**Gambar 2.5 Output kode program QueueLinkedList**

### 2.4.1 Analisis

Output tersebut menunjukkan bahwa program *queue* menggunakan Java dengan implementasi LinkedList telah berjalan dengan benar sesuai konsep *First In First Out (FIFO)*. Pada awalnya, elemen 0, 1, 2, 3, 4, dan 5 berhasil dimasukkan ke dalam antrian, yang ditampilkan dalam bentuk list [0, 1, 2, 3, 4, 5], di mana 0 merupakan elemen terdepan. Ketika dilakukan operasi *remove*, elemen paling depan yaitu 0 dihapus dari antrian, sehingga isi queue berubah menjadi [1, 2, 3, 4, 5]. Selanjutnya, operasi *peek* menampilkan elemen terdepan tanpa menghapusnya, yaitu 1. Terakhir, operasi *size* menunjukkan jumlah elemen yang tersisa dalam antrian sebanyak 5. Hal ini membuktikan bahwa queue bekerja sesuai prinsip FIFO, di mana elemen yang pertama masuk akan menjadi yang pertama keluar.

## 2.5 Praktikum class "ReverseData"

```java
package pekan4_2511533014;
import java.util.LinkedList;

public class ReverseData_2511533014 {
    public static void main (String[] args) {
        Queue<Integer> q = new LinkedList<Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("sebelum reverse" + q);
        Stack<Integer>s = new Stack<Integer>();
        while (!q.isEmpty()) { //Q -> Q
            s.push(q.remove());
        }
        while (!s.isEmpty()) { //S -> Q
            q.add(s.pop());
        }
        System.out.println("sesudah reverse= " + q); // [3, 2, 1 ]

    }
}
```

**Gambar 2.6 Kode program praktikum ReverseData**

Program ini merupakan implementasi pembalikan data (*reverse*) pada struktur *queue* menggunakan bantuan *stack* dalam bahasa Java. Data awal dimasukkan ke dalam queue, kemudian dipindahkan ke dalam stack sehingga urutannya terbalik (karena stack menggunakan konsep *LIFO*). Setelah itu, data dari stack dimasukkan kembali ke queue sehingga urutan elemen queue menjadi terbalik dari semula. Program ini menunjukkan kombinasi konsep *FIFO* (queue) dan *LIFO* (stack) untuk membalik urutan data.

Berikut penjelasan langkah-langkah kerjanya:

1. import java.util.LinkedList;
   import java.util.Queue;
   import java.util.Stack;
   → Mengimpor library:
   - LinkedList untuk implementasi queue
   - Queue sebagai interface antrian
   - Stack sebagai struktur data tumpukan
2. Queue<Integer> q = new LinkedList<Integer>();
   → Membuat objek queue dengan tipe Integer.
3. q.add(1);
   q.add(2);
   q.add(3);
   → Menambahkan elemen 1, 2, dan 3 ke dalam queue.
4. System.out.println("sebelum reverse" + q);
   → Menampilkan isi queue sebelum dibalik (output: [1, 2, 3]).
5. Stack<Integer> s = new Stack<Integer>();
   → Membuat objek stack untuk membantu proses pembalikan.
6. while (!q.isEmpty()) {
   s.push(q.remove());
   }
   → Selama queue tidak kosong:
   - Ambil elemen dari queue (remove)
   - Masukkan ke stack (push)

→ Hasilnya urutan menjadi terbalik di stack.

7. while (!s.isEmpty()) {
   q.add(s.pop());
   }
   → Selama stack tidak kosong:
   - Ambil elemen dari stack (pop)
   - Masukkan kembali ke queue (add)
     → Queue sekarang berisi data yang sudah terbalik.
8. System.out.println("sesudah reverse= " + q);
   → Menampilkan isi queue setelah dibalik (output: [3, 2, 1]).

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.7 dibawah ini.

```
<terminated> ReverseData_2511533014
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

**Gambar 2.7 Output kode program ReverseData**

### 2.5.1 Analisis

Output tersebut menunjukkan bahwa proses pembalikan data (*reverse*) pada struktur *queue* menggunakan bantuan *stack* dalam bahasa Java telah berjalan dengan benar. Pada awalnya, queue berisi elemen [1, 2, 3] yang ditampilkan sebagai kondisi sebelum dilakukan proses reverse. Kemudian, seluruh elemen dipindahkan ke dalam stack sehingga urutannya terbalik, lalu dikembalikan lagi ke queue. Hasil akhirnya ditampilkan sebagai [3, 2, 1], yang membuktikan bahwa urutan data berhasil dibalik. Hal ini terjadi karena kombinasi konsep *FIFO* pada queue dan *LIFO* pada stack, di mana elemen terakhir yang masuk ke stack akan keluar lebih dulu, sehingga menghasilkan urutan yang terbalik saat dimasukkan kembali ke queue.

## 2.6 Praktikum class "IterasiQueue"

```java
package pekan4_2511533014;
import java.util.Iterator;
public class IterasiQueue_2511533014 {
    public static void main (String[] args)
    {
        Queue<String> q = new LinkedList<>();

        q.add("Praktikum");
        q.add("Struktur");
        q.add("Data");
        q.add("Dan");
        q.add("Algoritma");
        Iterator<String> iterator = q.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }
}
```

**Gambar 2.8 Kode program praktikum IterasiQueue**

Program ini merupakan implementasi iterasi (penelusuran) elemen pada struktur data *queue* menggunakan *iterator* dalam bahasa Java. Data berupa beberapa string dimasukkan ke dalam queue yang diimplementasikan dengan LinkedList, kemudian ditelusuri satu per satu menggunakan objek *Iterator*. Setiap elemen dicetak secara berurutan sesuai urutan masuknya data, sehingga tetap mengikuti konsep *FIFO (First In First Out)*.

Berikut penjelasan langkah-langkah kerjanya:

1. **import java.util.Iterator;**
   **import java.util.Queue;**
   **import java.util.LinkedList;**
   → Mengimpor library:
   - Iterator untuk menelusuri elemen
   - Queue sebagai interface antrian
   - LinkedList sebagai implementasi queue
2. **Queue<String> q = new LinkedList<>();**
   → Membuat objek queue bertipe String.
3. **q.add("Praktikum");**
   **q.add("Struktur");**
   **q.add("Data");**

**q.add("Dan");**

**q.add("Algoritma");**

→ Menambahkan beberapa elemen string ke dalam queue sesuai urutan.

4. **Iterator<String> iterator = q.iterator();**

   → Membuat objek iterator untuk menelusuri isi queue.

5. **while (iterator.hasNext()) {**

   → Perulangan selama masih ada elemen berikutnya.

6. **System.out.print(iterator.next() + " ");**

   → Mengambil dan mencetak elemen satu per satu dari queue.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.9 dibawah ini

```
<terminated> IterasiQueue_2511533014 [Java Applic
Praktikum Struktur Data Dan Algoritma
```

**Gambar 2.9 Output kode program IterasiQueue**

### 2.6.1 Analisis

Output tersebut menunjukkan bahwa proses iterasi pada struktur data *queue* menggunakan *iterator* dalam bahasa Java telah berjalan dengan benar. Elemen yang dimasukkan ke dalam queue yaitu "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma" berhasil ditampilkan secara berurutan sesuai urutan masuknya data. Hal ini membuktikan bahwa iterasi menggunakan *Iterator* pada queue mempertahankan prinsip *FIFO (First In First Out)*, di mana elemen pertama yang dimasukkan akan ditampilkan terlebih dahulu. Selain itu, penggunaan perulangan while (iterator.hasNext()) memastikan semua elemen dalam queue ditelusuri dan dicetak tanpa ada yang terlewat.

## 2.7 Tugas Praktikum

### 2.7.1 “AntrianLoket_2511533014”

```java
package pekan4_2511533014;
import java.util.Stack;
public class AntrianLoket_2511533014 {
    String queue_3014[];
    int front_3014;
    int rear_3014;
    int max_3014;

    // Constructor
    public AntrianLoket_2511533014(int max) {
        this.max_3014 = max;
        queue_3014 = new String[max];
        front_3014 = 0;
        rear_3014 = -1;
    }

    // Cek kosong
    boolean isEmpty_3014() {
        return (rear_3014 < front_3014);
    }

    // Cek penuh
    boolean isFull_3014() {
        return (rear_3014 == max_3014 - 1);
    }

    // Enqueue
    void enqueue_3014(String data) {
        if (isFull_3014()) {
            System.out.println("Antrian penuh!");
        } else {
            rear_3014++;
            queue_3014[rear_3014] = data;
            System.out.println("Data berhasil ditambahkan ke antrian");
```

```java
        }
    }

    // Dequeue
    void dequeue_3014() {
        if (isEmpty_3014()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue_3014[front_3014] + " telah dilayani");
            front_3014++;
        }
    }

    // Display
    void display_3014() {
        if (isEmpty_3014()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no = 1;
            for (int i = front_3014; i <= rear_3014; i++) {
                System.out.println(no + ". " + queue_3014[i]);
                no++;
            }
        }
    }

    // Reverse
    void reverse_3014() {
        if (isEmpty_3014()) {
            System.out.println("Antrian kosong!");
            return;
        }

        Stack<String> stack = new Stack<>();
```

```java

        // Queue → Stack
        for (int i = front_3014; i <= rear_3014; i++) {
            stack.push(queue_3014[i]);
        }

        // Stack → Queue
        int i = front_3014;
        while (!stack.isEmpty()) {
            queue_3014[i] = stack.pop();
            i++;
        }

        System.out.println("Antrian berhasil dibalik");
    }
}
```

Program AntrianLoket_2511533014 merupakan implementasi struktur data queue (antrian) menggunakan array dengan konsep FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Program ini memiliki variabel untuk menyimpan data (queue_3014), posisi depan (front_3014), posisi belakang (rear_3014), dan kapasitas maksimum (max_3014). Selain itu, terdapat method penting seperti enqueue_3014() untuk menambah data, dequeue_3014() untuk menghapus data, display_3014() untuk menampilkan isi antrian, serta reverse_3014() untuk membalik urutan antrian menggunakan bantuan struktur data stack.

Berikut penjelasan langkah-langkah kerjanya:

1. Deklarasi Atribut

• String queue_3014[];

• int front_3014;

• int rear_3014;

• int max_3014;

→ Atribut untuk menyimpan data antrian, posisi depan, posisi belakang, dan kapasitas maksimum queue.

2. Konstruktor

public AntrianLoket_2511533014(int max)

→ Konstruktor untuk menginisialisasi objek queue saat dibuat.

• this.max_3014 = max;

→ Mengisi nilai kapasitas maksimum queue.

• queue_3014 = new String[max];

→ Membuat array sebagai tempat penyimpanan data.

• front_3014 = 0;

→ Menentukan posisi awal antrian (depan).

• rear_3014 = -1;

→ Menentukan posisi awal belakang antrian (kosong).

3. Method isEmpty_3014()

• boolean isEmpty_3014()

→ Mengecek apakah antrian kosong.

• return (rear_3014 < front_3014);

→ Mengembalikan nilai true jika tidak ada data dalam antrian.

4. Method isFull_3014()

• boolean isFull_3014()

→ Mengecek apakah antrian penuh.

• return (rear_3014 == max_3014 - 1);

→ Mengembalikan nilai true jika array sudah penuh.

5. Method enqueue_3014()

• void enqueue_3014(String data)

→ Menambahkan data ke dalam antrian (bagian belakang).

• if (isFull_3014())

→ Mengecek apakah antrian penuh.

• rear_3014++;

→ Menambah posisi belakang.

• queue_3014[rear_3014] = data;

→ Menyimpan data ke dalam array.

6. Method dequeue_3014()

• void dequeue_3014()

→ Menghapus data dari bagian depan antrian.

• if (isEmpty_3014())

→ Mengecek apakah antrian kosong.

• System.out.println(queue_3014[front_3014] + " telah dilayani");

→ Menampilkan data yang dihapus.

• front_3014++;

→ Menggeser posisi depan.

7. Method display_3014()

• void display_3014()

→ Menampilkan seluruh isi antrian.

• for (int i = front_3014; i <= rear_3014; i++)

→ Melakukan perulangan untuk menampilkan data.

→ Data ditampilkan berurutan dari depan ke belakang.

8. Method reverse_3014()

• void reverse_3014()

→ Membalik urutan isi antrian.

• Stack<String> stack = new Stack<>();

→ Membuat stack sebagai bantuan.

• stack.push(queue_3014[i]);

→ Memasukkan data queue ke stack.

• queue_3014[i] = stack.pop();

→ Mengambil kembali data dari stack ke queue (dalam urutan terbalik).

→ Menghasilkan antrian yang sudah dibalik.

### 2.7.2 “AntrianLoketDriver”

```java
package pekan4_2511533014;
import java.util.Scanner;
public class AntrianLoketDriver_2511533014 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLoket_2511533014 q = new AntrianLoket_2511533014(10);

        int pilihan;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input.nextLine();
                    q.enqueue_3014(nama);
                    break;

                case 2:
                    q.dequeue_3014();
                    break;

                case 3:
                    q.display_3014();
                    break;
```

```java

                case 4:
                    q.reverse_3014();
                    q.display_3014();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan != 5);

        input.close();
    }
}
```

Program AntrianLoketDriver_2511533014 merupakan kelas utama (driver) yang digunakan untuk menjalankan dan menguji program antrian. Program ini menggunakan input dari pengguna melalui Scanner untuk memilih menu operasi seperti menambah antrian, menghapus antrian, menampilkan isi antrian, serta membalik urutan antrian. Kelas ini berinteraksi langsung dengan objek AntrianLoket_2511533014 yang telah dibuat sebelumnya, sehingga semua proses manipulasi data antrian dilakukan melalui method yang ada pada kelas tersebut.

Berikut penjelasan langkah-langkah kerjanya:

1. Inisialisasi Objek dan Variabel

• `Scanner input = new Scanner(System.in);`

→ Membuat objek Scanner untuk membaca input user.

• `AntrianLoket_2511533014 q = new AntrianLoket_2511533014(10);`

→ Membuat objek antrian dengan kapasitas maksimal 10 data.

• `int pilihan;`

→ Variabel untuk menyimpan pilihan menu dari user.

2. Perulangan Menu (do-while)

• `do { ... } while (pilihan != 5);`

→ Perulangan yang terus berjalan sampai user memilih keluar (menu 5).

→ Digunakan agar program terus menampilkan menu.

3. Tampilan Menu

• `System.out.println("=== PROGRAM ANTRIAN LOKET ===");`

→ Menampilkan judul program.

• Pilihan 1–5

→ Menampilkan daftar menu yang bisa dipilih user.

4. Input Pilihan

• `pilihan = input.nextInt();`

→ Mengambil input angka dari user.

• `input.nextLine();`

→ Membersihkan buffer agar input berikutnya tidak terlewat.

5. Percabangan (Switch Case)

• `switch (pilihan)`

→ Menentukan aksi berdasarkan pilihan user.

6. Case 1 (Tambah Antrian)

• `String nama = input.nextLine();`

→ Mengambil input nama pelanggan.

• `q.enqueue_3014(nama);`

→ Menambahkan data ke antrian.

7. Case 2 (Hapus Antrian)

• `q.dequeue_3014();`

→ Menghapus data dari antrian (FIFO).

8. Case 3 (Tampilkan Antrian)

• `q.display_3014();`

→ Menampilkan seluruh isi antrian.

9. Case 4 (Reverse Antrian)

• `q.reverse_3014();`

→ Membalik urutan antrian.

• `q.display_3014();`

→ Menampilkan hasil setelah dibalik.

10. Case 5 (Keluar)

• `System.out.println("Program selesai");`

→ Menghentikan program.

11. Default Case

• `System.out.println("Pilihan tidak valid!");`

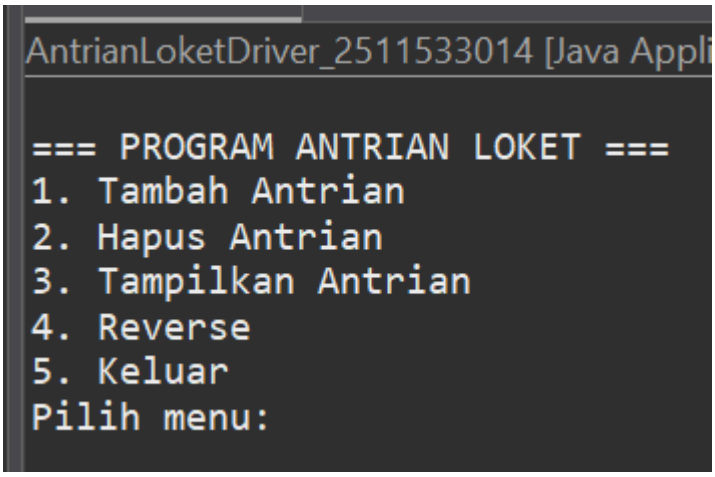
→ Menampilkan pesan jika input tidak sesuai menu.

12. Menutup Scanner

• `input.close();`

→ Menutup penggunaan Scanner untuk menghindari memory leak.

### 2.7.3 Menu Program




### 2.7.4 Output

```
<terminated> AntrianLoketDriver_2511533014 [Java Application]

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian
```

```
<terminated> AntrianLoketDriver_2511533014 [Java Applic
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. Budi
2. Andi
```

```
<terminated> AntrianLoketDriver_2511533014 [Ja
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

```
<terminated> AntrianLoketDriver_2511533014 [Java Applic
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. Budi
2. Andi
```

### 2.7.5 Analisis

Output program menunjukkan bahwa struktur data queue berjalan sesuai konsep FIFO (First In First Out), di mana data ditambahkan dari belakang (enqueue) dan dihapus dari depan (dequeue). Awalnya pengguna menambahkan “Andi” dan “Budi” sehingga antrian berisi dua data berurutan. Saat ditampilkan, urutan masih sesuai input. Ketika dilakukan proses reverse, urutan antrian berubah menjadi terbalik (Budi di depan, Andi di belakang), yang membuktikan bahwa method reverse_3014() bekerja dengan bantuan stack. Setelah itu, saat dilakukan dequeue, data paling depan yaitu “Budi” dihapus terlebih dahulu. Terakhir, ketika ditampilkan kembali, hanya tersisa “Andi” dalam antrian, yang menandakan seluruh operasi (enqueue, display, reverse, dequeue) berjalan dengan benar sesuai logika program.

# BAB III
# PENUTUP

## 3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data queue (antrian) bekerja dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Implementasi queue menggunakan array berhasil dilakukan dengan memanfaatkan variabel front dan rear untuk mengatur posisi data. Method yang dibuat seperti enqueue, dequeue, display, isEmpty, dan isFull dapat berjalan dengan baik sesuai fungsinya. Selain itu, penambahan fitur reverse juga berhasil membalik urutan antrian dengan bantuan stack. Secara keseluruhan, program berjalan dengan benar tanpa error dan mampu merepresentasikan konsep antrian dalam pemrograman secara jelas.

# DAFTAR PUSTAKA

[1] GeeksforGeeks, “Queue Data Structure,” GeeksforGeeks, 2024. [Online]. Available: https://www.geeksforgeeks.org/queue-data-structure/. [Accessed: 30-Apr-2026].

[2] Oracle, “Queue Interface in Java,” Oracle Documentation, 2024. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html. [Accessed: 30-Apr-2026].

[3] Programiz, “Queue Data Structure,” Programiz, 2024. [Online]. Available: https://www.programiz.com/dsa/queue. [Accessed: 30-Apr-2026].

[4] Tutorialspoint, “Data Structure - Queue,” TutorialsPoint, 2024. [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm. [Accessed: 30-Apr-2026].